Vivid 2.0

1992. március 14.

Copyright 1989–1992 Stephen B. Coy

Magyar fordítás

Tartalomjegyzék

- Bevezetés
- A 2.0-s verzió változásai
- A Vivid futtatása
- A statisztikai kijelzés
- Jogi tudnivalók
- Köszönetnyilvánítás
- A bemeneti fájl formátuma
  - Előfeldolgozó
  - A stúdió
  - Fényforrások
  - Felületek
    - Bump mapping (felületi egyenetlenség)
    - Térbeli (solid) textúra
    - Mandelbrot
  - Geometriai primitívek: gömb, gyűrű, sokszög, háromszögfolt, kúp
  - Transzformációk
  - Vágás (clipping)
- Segédprogramok: img2gif.exe, paste.exe, up.exe, down.exe
- Fájlformátumok
- Irodalomjegyzék

Bevezetés a Vivid 2.0-hoz

Meglepetés! Igen, végre itt a 2.0. Elnézést kérek mindazoktól, akik hónapok óta várnak a megjelenésére. A diplomamunkámmal voltam elfoglalva, és persze azzal, hogy „még ezt az egy funkciót” beépítsem. Közben néhány hibát is agyoncsaptam, bár biztos vagyok benne, hogy az újak már szaporodnak a falakban, és csak erre a kiadásra várnak. A hibák ellenére is a Vivid-et tartom az egyik legjobb sugárkövetőnek, amit (én) valaha írtam.

A kapott archívumban kell lennie ennek a dokumentumnak, a Vivid futtatható állományának, néhány minta bemeneti fájlnak, valamint az elkészült kép utólagos feldolgozására szolgáló eszközöknek. A Vivid.exe bármely MS-DOS alapú, legalább 286-os processzorral és matematikai koprocesszorral rendelkező rendszeren fut. A sugárkövetés rendkívül lassú lehet, és ennél gyengébb gépen tényleg nem éri meg. Ha mégsem hisz nekem, írjon nyugodtan, és küldök egy csupasz 8086-ra fordított példányt. Remélem, a következő egy hónapban nem akarja használni a PC-jét. Vissza a 286/287-es verzióhoz: ez a változat DOS-extendert is tartalmaz, így a Vivid ki tudja használni a gépben lévő kiterjesztett (extended) memóriát. Akik ennél is jobbat szeretnének, azoknak 386-os változatok is elérhetők – lásd a Jogi tudnivalók fejezetet.

A dokumentum további része részletesen leírja a Vivid és a segédeszközök futtatását, valamint a bemeneti fájl felépítését. Javaslom, hogy a Vivid tanulásának kezdetén nyomtassa ki ezt a fájlt és néhány mintafájlt. Egy példa többet ér ezer szónál – vagy legalább annyit, mint a papír, amelyre nyomtatták.

A 2.0-s verzió (legtöbb) változása

Megtanultam helyesen leírni Mark VandeWettering nevét. Bocs az 1.0 miatt, Mark.

A belül/kívül problémák megoldódtak, így már nincs szükség negatív sugarú gömbökre, és nem kell az óramutató szerinti vagy azzal ellentétes sokszög-körüljárás miatt aggódni.

A sokszögfoltok (polygon patch) sokkal egészségesebbek lettek.

Visszadugtam az agyamat. Az 1.0-s dokumentációban a „fuzzy” (elmosódott) gömbök leírása hibás volt. Most már jobb.

A mélységélesség (depth of field) opció használatakor egy új „samples” paraméterrel szabályozható a pixelenként kilőtt sugarak száma.

A gömb alakú fényforrásoknál is megadható a „samples” paraméter, amely a feléjük kilőtt árnyéksugarak számát határozza meg.

A fényforrások számát már csak a memória és az Ön türelme korlátozza. A korlát korábban 20 volt, de ez az információ sosem került be az 1.0 dokumentációjába. Az objektumok számára vonatkozó korlát is megszűnt, bár az operációs rendszer miatt gyakorlatilag kb. 2000 objektumos határ továbbra is fennáll. A DOS-extenderes változatot csak a memória korlátozza.

Megadható az egy befoglaló dobozban (bounding box) lévő objektumok száma. Az alapértelmezés, a 4, megegyezik a korábbi rögzített értékkel. A stúdió-struktúrába írt

```
bunching 6
```

sorral állítható. Az érték módosítása befolyásolja az automatikus befoglalódoboz-építés teljesítményét. Az eredmény képfüggő, de szerintem a 4 jó alapérték.

A kameramodell egy hibája miatt torzult a kép, ha a stúdió-struktúrában megadott „up” vektor nem volt merőleges a nézési irányra. Ez javítva lett.

A hagyományos sík vetítési modellhez nem egy, nem kettő, hanem rögtön három új vetítési modell csatlakozott: a gömbi (spherical), az ortografikus (orthographic) és a parallaxis (parallax). Részletek a stúdió-struktúráról szóló részben.

A bemeneti fájl formátuma megváltozott: már nincs szükség pontosvesszőkre és egyenlőségjelekre. A régi formátummal való kompatibilitás érdekében továbbra is elfogadja őket, de szóközként kezeli.

Gyors renderelési mód került be az előnézetek gyorsítására.

Az értelmező (parser) egyszerű matematikai műveleteket is támogat, számokon és vektorokon (rgb hármasokon és xyz koordinátákon). A támogatott vektorműveletek: vektoriális szorzat, összeadás, kivonás, skálázás és skaláris szorzat. A numerikus műveletek: szorzás, osztás, összeadás, kivonás és hatványozás. Támogatott továbbá a szinusz, koszinusz, tangens, arkusz szinusz, arkusz koszinusz, arkusz tangens és a négyzetgyök függvény.

Hengert most már úgy is megadhat, hogy a kúpnak egyetlen sugár paramétert ad.

Megérkezett az előfeldolgozó: include fájlok, define-ok, undef-ek, és megszűntek a rögzített színek, helyettük a color.vc szolgál.

Támogatottak a globális transzformációk. Az include fájlokkal együtt ez jelentősen megkönnyíti a jelenetmodellezést.

A primitívek síkokkal, gömbökkel vagy kúpokkal vághatók, így összetettebb modellek is egyszerűbben készíthetők.

A háttérre a korábbi egyetlen szín helyett paletta is leképezhető.

Egy csomó új parancssori kapcsoló érkezett. Ha a parancssorba argumentumok nélkül beírja, hogy vivid, megkapja a listájukat.

A „noise” kulcsszó neve „turbulence” lett, hogy jobban kövesse a szakirodalmat.

A Vivid futtatása

A Vivid bemeneti fájljai .v, a kimeneti fájlok .img kiterjesztésűek. Az előfeldolgozónak köszönhetően include fájlok is támogatottak; ezek kiterjesztése tetszőleges lehet. Egy fájl bemeneti fájlba illesztésének szintaxisa:

```
#include fájlnév
```

Az include fájlok kiterjesztése tetszőleges. Az alábbi lista az általam rendszeresen használt kiterjesztéseket tartalmazza. Ha mindannyian ugyanazt a konvenciót követjük, könnyebb lesz a bemeneti fájlok cseréje. Ha Önnek más konvenció kényelmesebb, nyugodtan alakítson ki sajátot.

```
.v     Egy kép fő bemeneti fájlja. Kötelező.
.img   24 bites kimeneti fájl.
.map   Palettafájl az img2gif-hez vagy háttér-leképezéshez.
.vc    Színdefiníciókat tartalmazó include fájl. Lásd color.vc.
.vo    Objektumdefiníciót tartalmazó include fájl.
.vs    Felületdefiníciókat tartalmazó include fájl.
```

A Vivid által készített .img fájlok teljes 24 bites színmélységűek. Formátumukat a dokumentum későbbi része ismerteti. Mivel a legtöbb rendszer nem tud 24 bites grafikát

megjeleníteni, mellékeltem egy eszközt, amely az .img fájlokat vpic-kel vagy cshow-val megjeleníthető .gif fájlokká alakítja. Ennek neve img2gif, működését szintén később részletezem. Általában egy kép elkészítése így néz ki:

```
1) Készítsen egy test.v bemeneti fájlt kedvenc szövegszerkesztőjével.
2) Futtassa a Vivid-et.
     C>vivid test
3) Várjon, amíg elkészül. Ez a kép bonyolultságától és a gép
   sebességétől függően néhány perctől néhány napig tarthat.
   Remek alkalom a hűtő kifosztására.
4) Alakítsa át az eredményt (test.img) .gif fájllá.
     C>img2gif -m -d test
   A -m kapcsoló median cut algoritmussal határozza meg a palettát,
   a -d bekapcsolja a Floyd–Steinberg ditheringet.
5) Nézze meg a képet.
     C>vpic test.gif
```

Az előfeldolgozó megvalósítása miatt a Vivid egy xyzzy.v nevű ideiglenes fájlt hoz létre a lemezen. Jelenleg a Vivid a használat után nem törli ezt a fájlt. Azért hagyom meg, mert néha hasznos belenézni, ha ki akarjuk deríteni, mit művel az előfeldolgozó a bemeneti definícióval. Bármikor nyugodtan törölheti.

A Vivid számos parancssori kapcsolót támogat, amelyekkel a bemeneti fájl módosítása nélkül változtatható a program működése. A jelenleg támogatott kapcsolók:

```
-s        Csendes mód, azaz nincs statisztikai kijelzés.
-r        Folytatás (resume).
-i x y    A képméret beállítása x × y pixelre.
-n        Nincsenek árnyékok, ugyanaz, mint a stúdió no_shadows jelzője.
-d #      A maximális rekurziós mélység beállítása.
-a mód    Élsimítási mód beállítása. Érvényes módok: none, quick,
          corners és adaptive. Mivel nem tudok gépelni, elég a mód
          első betűjét megadni.
-p        Ne használja az előfeldolgozót.
```

A Vivid képes egy megszakított kép generálását folytatni. Ha a generálás közben le kell állítania, a Ctrl-C vagy Ctrl-BREAK hatására a Vivid a következő képsor végén megáll. Lassú képnél ez akár néhány percig is eltarthat. Ez csak akkor működik, ha a statisztika engedélyezve van (alapértelmezés). Később a generálás a -r kapcsolóval folytatható.

Statisztika

Meglepő módon egyeseket érdekel a statisztika, amelyet a Vivid feldolgozás közben kijelez. Valahogy az a gondolatuk támadt, hogy a számok hasznos információt hordozhatnak, ha valahol dokumentálva lennének. Kiderült a titok: semmit sem jelentenek. Csak azért tettem be őket, mert szeretem nézni, ahogy változnak, miközben a képre várok. Bocs, hogy kipukkasztom a buborékot. Na jó, csak vicceltem, most boldog?

Indításkor a Vivid kiírja az aktuális verziószámot és a szerzői jogi megjegyzést. Ezután megjeleníti a bemeneti fájlok nevét, ahogy az előfeldolgozó feldolgozza őket. Majd a fényforrások és primitívek folyamatosan növekvő összesítése jelenik meg, miközben az értelmező beolvassa az előfeldolgozott bemeneti fájlt. Amikor a fájl teljesen beolvasásra került, meghívódik az „automágikus” befoglalódoboz-kód. Ahogy a befoglaló dobozok létrejönnek, a primitívek száma nő. A befoglaló dobozok elkészülte után kiíródik a jelenet kiterjedése: az összes primitív minimális és maximális értéke tengelyenként. Ekkor kezdődik az első képsor renderelése. Amikor ez elkészül, a képernyő törlődik, és megjelenik az aktuális statisztika teljes kijelzése. Az egyes számok jelentését most minden homályosítási kísérlettől mentes, dicsőséges részletességgel ismertetem. Remélem, érthető lesz.

A felbontás (resolution) a végső kép mérete pixelben. A mellette látható sor (line) a sugárkövető előrehaladását mutatja. Például egy 320 pixel széles és 200 pixel magas képnél a felbontás 320x200 lesz (hihetetlen!), a sor értéke pedig 0-tól 200-ig nő. Gyors módban ez a szám 1 helyett 6-osával nő.

A következő négy sor a kilőtt sugarak számáról tájékoztat. A szemsugarak (eye rays) a nézőpontból a jelenetbe kilőtt sugarak. A visszavert sugarakat (reflected) a tükröző felületek, a megtört sugarakat (refracted) az átlátszó felületek hozzák létre. Az összeg (total) egyszerűen ezek összege. Ha a bemeneti fájlban nincs tükröző vagy átlátszó felület, ezek a számok 0-n maradnak.

A következő rész az árnyéksugarakról szól. Az árnyéksugarakat a metszéspontból lőjük ki az egyes fényforrások felé, hogy kiderüljön, takarja-e más primitív a fényt (azaz vet-e árnyékot). A Vivid árnyék-gyorsítótárat (shadow cache) használ a folyamat gyorsítására. A találat (cache hit) azt jelenti, hogy a gyorsítótárban lévő objektum árnyékot vetett, így nem kellett sugarat kilőni. A gyorsítótár-százalék egyszerűen a találatok száma osztva az árnyéksugarak számával.

A következő szám a pixelenként kilőtt sugarak átlagos száma. Ebből elég jól látható, mennyire dolgozik keményen a sugárkövető a képen. Élsimítás, tükröződés és fénytörés nélkül ennek 1,0-nak kell lennie. Az adaptív élsimítás akár 16-ig is felviheti, attól függően, mennyi extra munkát végez. A visszavert és megtört sugarak tovább növelik. Gyors módban akár kb. 0,066-ig is lemehet. Ha a szám 1-nél kisebb, az azt jelenti, hogy a sugárkövető egyes pixeleket megbecsül ahelyett, hogy ténylegesen sugarat lőne ki értük.

A sugaranként átlagos sorba állítások száma (average queues per ray) a befoglalási sémával kapcsolatos. Az automatikus befoglalódoboz-generátor fa szerkezetű hierarchiát épít a befoglaló dobozokból, és a rendszer minden sugarat ezekkel tesztel. Ha egy dobozt metsz a sugár, a dobozban lévő objektumok (további befoglaló dobozok vagy primitívek) bekerülnek a prioritási sorba további metszésvizsgálatra. A „bounds checked” szám azt mutatja, hány befoglalódoboz-metszésvizsgálat történt. A „queue inserts” azt, hány objektum ment át a befoglalási vizsgálaton és került a prioritási sorba. A „queue resets” azt, összesen hányszor nullázódott a prioritási sor; ennek meg kell egyeznie a sugarak összes számának, az árnyéksugarak számának és a gyorsítótár-találatok számának összegével. Ne kérdezze, miért – egyszerűen így lesz. A „max queue size” a sorban renderelés közben egyidejűleg lévő objektumok maximális számát mutatja.

Az utolsó szám, a maximális rekurziós mélység, azt mutatja, legfeljebb hány szint mélyre kellett lemennie a sugárkövetőnek a kép renderelése során. A perjel utáni szám a megengedett maximális mélység.

Jogi tudnivalók

A Vivid szerzői joga: Copyright 1989–1992, Stephen B. Coy. Minden jog fenntartva.

Ez a csomag teljes egészében szabadon terjeszthető. Ennek során legfeljebb jelképes másolási díj kérhető. A csomag egyetlen része sem építhető be kereskedelmi csomagba kifejezett írásos engedély nélkül. A Vivid vagy a segédeszközei kereskedelmi terjesztésével kapcsolatos kérdésekkel az alábbi címen érhető el:

```
Stephen Coy
Vivid Software
15205 NE 13th Pl. #2904
Bellevue, WA 98007
(206) 641-8615

INTERNET:   coy@ssc-vax.boeing.com  vagy  uw-beaver!ssc-vax!coy
CompuServe: 70413,136
```

Eredetileg sharewareként akartam terjeszteni a Vivid-et, de nagyon nem szeretem a lelkiismeret-furdaltató játékokat, amelyeket a legtöbb shareware-szerző beépít. Ezért a „koldusware” (beggarware) mellett döntöttem. Itt a termék. Használja. Ha tetszik, nagyon örülnék, ha küldene egy 30 dolláros csekket. Ha a 30 dollárt túlzásnak tartja, küldjön kevesebbet (vagy többet), vagy sört. Ha úgy gondolja, a Vivid fabatkát sem ér, legalább egy képeslapot küldjön, amelyben elmagyarázza, miért nem tetszik. Ha a 386-os változatokat szeretné, 50 dolláros szinten kell regisztrálnia. Sajnálom, de nehéz idők járnak, és a fordítóprogramok sem olcsók. Mindenesetre ez nagyjából egy átlagos mostani játék ára, úgyhogy nem tartom túlzásnak. Szokás szerint pénz helyett elfogadok nagyon szívhez szóló siránkozó történeteket, sört, szoftvert (kalózmásolatot nem, köszönöm), cserealapot és fiatal hölgyeket is. A 386-os lemezen a Vivid Zortech és Intel fordítóval készült változatai is megtalálhatók, így választhat a DOS-extenderek közül. Az Intel-változat DPMI-kompatibilis, és előnye, hogy virtuális memóriát támogat, így akár 128 MB lemezterület is lefoglalható memóriaként a komolyan óriási modellekhez. A Zortech-változat DPMI-, VCPI- és XMS-kompatibilis, és kevesebb helyet foglal. Úgy tűnik, kb. 500 KB-tal több memóriát is talál, mint az Intel-változat, viszont közvetlenül nem támogatja a virtuális memóriát. Ha egy másik, virtuális memóriát támogató DPMI-gazda aktív – például a 386 enhanced módban futó Windows 3.0 –, akkor a Zortech-változat is ki tudja használni.

Akár küld valamit, akár nem, szívesen hallanék bármilyen javaslatról vagy hibáról. Persze a pénzzel kísért kérések általában nagyobb prioritást kapnak. Minden levélre és e-mailre válaszolok, még ha időnként kicsit lassú is vagyok. Az e-mailre szoktam a leggyorsabban reagálni. Ha próbált elérni, de nem kapott választ, kérem, próbálja újra. Ha akkor sem kap választ, kénytelen lesz a hagyományos postához folyamodni.

A BBS-ezők bármelyik ADEnet-csomóponton keresztül elérhetnek. Az ADEnet BBS-ek hálózata, amely elsősorban az AutoCAD és más CAD-programok professzionális szintű támogatására szolgál. Emellett van egy animációs és renderelési konferencia is, ahol a Vivid-del vagy általában a grafikával kapcsolatos kérdésekre válaszolnak. Általában naponta bejelentkezem, így a válaszidő elég rövid. Egy másik BBS, ahova sokat járok, a TurboTech. Ray Johnson, a TurboTech sysopja és a TurboCit szerzője külön szobát biztosított a sugárkövetéssel kapcsolatos beszélgetéseknek, és rengeteg lemezterületet a képeknek. Nézze meg a beszélgetéseket is – a BBS érdekes embereket vonz. A mélyebb zsebűeknek: a CompuServe COMART fórumán is gyakran megfordulok.

Ray Johnson Turbo Tech BBS-e a (206) 362-6828-as számon érhető el.

Alacrity Design & Engineering Network (ADEnet)

Az ADEnet a tervező- és mérnöki közösséget szolgálja azzal, hogy a számukra hasznos információkat széles körben elérhetővé teszi. Ezt háromféleképpen teszi: az ADEnet üzenetkonferenciákkal, programok és adatfájlok megosztásával az ADEneten keresztül, valamint iparági hírek és sajtóközlemények terjesztésével.

Az ADEnet tagjainak listája

Az alábbi lista az ADEnet jelenlegi tag-BBS-eit tartalmazza. Minden ADEnet-tag egy helyi sysop által üzemeltetett, független BBS.

```
Alacrity Software BBS  206-643-5477    Bellevue, WA        DUAL (14.4)
The Graphics Alternative 510-524-2780  El Cerruto, CA      DUAL (14.4)
Off Broadway           510-547-5264    San Francisco, CA   V32
PC-AUG BBS             602-952-0638    Phoenix, AZ         DUAL (9600)
Mind Image BBS         612-781-1720    Minneapolis, MN     v22, MNP5
LABB                   717-394-1357    Lancaster, PA       v22, MNP5 [%]
AEC-ONLINE             818-360-7022    Los Angeles, CA     v22
The University BBS     908-544-8193    Shrewsbury Twp, NJ  DUAL (14.4)
AZCAD BBS              +61-3-481-6873  Ausztrália          v32bis
```

Minden bejegyzés tartalmazza a BBS nevét és (elsődleges) telefonszámát, majd a helyét. A bejegyzés végén a legnagyobb megengedett átviteli sebesség (bps) és a protokoll áll.

```
DUAL  - US Robotics Dual Standard, HST és v.32/v.42bis támogatással
HST   - US Robotics Courier HST
v32   - Ipari szabvány v.32/v.42bis, legfeljebb 9600 bps
v32b  - Ipari szabvány v.32bis/v.42bis, legfeljebb 14,4k bps
v22   - Ipari szabvány v.22, legfeljebb 2400 bps
MNP5  - Microcom Network Protocol Class 5 támogatás
v42b  - CCITT v.42bis támogatás

[*] A CAD/Engineering Services BBS 22:00–05:00 (CST) között fogad hívásokat.
[%] A LABB többvonalas rendszer, ebből csak a fővonal szerepel.
```

Az ADEnet jelenlegi üzenetkonferenciái:

```
Konferencia  Leírás
-----------------------------------------------------
Autodesk     Autodesk termékek támogatása
A/E/C        Építészet, mérnöki tervezés és építőipar
CAD/CAM      Gyártás, NC-programozás
Civil        Építőmérnöki tervezés
CompuGFX     Számítógépes grafika, renderelés és animáció
DTP          Kiadványszerkesztés
ForSale      Használt eszközök adás-vétele, állásajánlatok
Generic      Generic Software termékek támogatása
GIS          Földrajzi információs szolgáltatások
uStation     Intergraph Microstation támogatás
News         Iparági hírek és sajtóközlemények
Public       Hálózatszintű (ADEnet) általános konferencia
TechTalk     Műszaki hardver- és szoftverbeszélgetések
```

Csatlakozás az ADEnethez

Ha Ön egy BBS rendszergazdája, és szeretne csatlakozni az ADEnethez, töltse le az ADENET.ZIP fájlt. Ebben részletesebb bemutatást, jelentkezési lapot és minden szükséges információt és adatfájlt talál.

Az ADEnet jelenleg a PC-Relay, a QWK/REP és a FidoNet echomail üzenethálózati protokollokat támogatja. A PC-Relay-t vagy QWK/REP-et használó csomópontok számára az

Alacrity Software BBS az ADEnet központja (hub). Az echomailt használó csomópontok felé a The University BBS szolgál „hídként”.

Köszönetnyilvánítás

Elsőként Mark VandeWetteringnek kell köszönetet mondanom az MTV nevű sugárkövetőjéért. A Vivid egyes részei közvetlenül az MTV kódjából származnak. Remélhetőleg csak a hibamenteseket vettem át. Köszönöm Mike Thompsonnak különféle hozzájárulásait, mind a műszakiakat, mind az erjesztetteket. Köszönet Gregor Harrisonnak, aki sokat segített a lex és a yacc megértésében. Még mindig kissé össze vagyok zavarodva, de így szeretem. Köszönet Jason Osgoodnak (ADEnet), hogy fórumot biztosított a Vivid-del kapcsolatos dohogásaimnak. Köszönet Drew Wellsnek, aki a CompuServe-hez való csatlakozásra biztatott. És köszönet mindazoknak, akik a Usenetre írt bejegyzéseikkel tudtukon kívül segítették a Vivid-et. Végül köszönöm mindenkinek, aki időt szánt arra, hogy megjegyzéseket (és képeket, sört és pénzt) küldjön az 1-es verzióhoz. Ha az ember hazaérve nem csak számlákat, hanem egy kedves levelet és egy csekket talál a postaládában, az egész hete jobban telik. Köszönöm.

A GIF a CompuServe védjegye. Az MS-DOS a Microsoft védjegye.

A bemeneti fájl formátuma

A Vivid bemeneti fájljainak készítésekor tartsa szem előtt, hogy a Vivid értelmezője megkülönbözteti a kis- és nagybetűket. A makrók (lásd az Előfeldolgozó fejezetet) szintén kis- és nagybetű-érzékenyek.

A Vivid jobbsodrású koordináta-rendszert használ az objektumok térbeli helyének megadására. A jobbsodrású koordináta-rendszer úgy képzelhető el, hogy az x tengely jobbra, az y tengely a képernyő belseje felé, a z tengely pedig felfelé mutat. A jelenetfájl minden objektumának helyét ebben a koordináta-rendszerben egy x, y, z hármas határozza meg.

A színeket r, g, b hármas adja meg, ahol az egyes összetevők általában a 0..1 tartományba esnek. A színválasztás megkönnyítésére a Vivid tartalmazza a color.vc fájlt, amelyben néhány tucat előre definiált szín található. Nevük és rgb-értékük a color.vc megtekintésével vagy kinyomtatásával tudható meg. Ha ezt a fájlt a bemeneti fájl elején #include-dal beilleszti, ezek a nevek bárhol használhatók, ahol rgb hármast vár a program.

Az értelmező egyszerű matematikai műveleteket is támogat, számokon és vektorokon (rgb hármasokon és xyz koordinátákon). A támogatott vektorműveletek: vektoriális szorzat, összeadás, kivonás, skálázás és skaláris szorzat. A numerikus műveletek: szorzás, osztás, összeadás, kivonás és hatványozás. Támogatott továbbá a szinusz, koszinusz, tangens, arkusz szinusz, arkusz koszinusz, arkusz tangens és a négyzetgyök függvény.

```
Vektorműveletek
---------------
a b c cross x y z    vektoriális szorzat, eredménye vektor
a b c dot x y z      skaláris szorzat, eredménye szám
a b c + x y z        vektorösszeadás, eredménye vektor
a b c - x y z        vektorkivonás, eredménye vektor
a b c * n            vektor skálázása n-nel
a b c / n            vektor skálázása 1/n-nel
-(a b c)             vektor ellentettje

Numerikus műveletek
-------------------
sin(x), cos(x), tan(x)   trigonometrikus függvények
asin(x), acos(x), atan(x)
sqrt(x)                  x négyzetgyöke
pow(x y)                 x az y-adik hatványon
*, /, +, -               szokásos matematikai műveletek
```

Megjegyzendő, hogy ezek a műveletek bizonyos kétértelműséget visznek a bemeneti nyelvbe. A problémát súlyosbítja, hogy az értelmező csak egy tokennel tud előrenézni, amikor eldönti, hogyan kezelje az aktuális bemenetet. Arra biztatom, hogy a zárójelek bőkezű használatával kerülje el az ilyen problémákat, és szüntessen meg minden lehetséges kétértelműséget az egyenleteiben. Például:

```
n * a b c   eredménye lehet
(n*a) b c   vagy
(n*a) (n*b) (n*c)
```

attól függően, milyen kedvében van aznap az értelmező.

Ha nagyon akarja, biztosan ki tudná deríteni, mit csinál az értelmező, de ez a jövőben változhat, úgyhogy kímélje meg magát a fáradságtól, és egyszerűen tegye ki a zárójeleket.

A bemeneti fájl megjegyzéseket is tartalmazhat. A programok megjegyzéseihez hasonlóan a Vivid bemeneti fájljainak megjegyzéseit az értelmező figyelmen kívül hagyja. A Vivid megjegyzései a C++ szintaxisát követik. A többsoros megjegyzés /*-gal kezdődik és */-gal végződik; a határolók közötti tartalmat a program figyelmen kívül hagyja. Az egysoros megjegyzés //-rel kezdődik és a sor végéig tart. Például:

```
/* Ez egy megjegyzés,
   amely több soron át tart */

// Ez egy egysoros megjegyzés
```

A megjegyzéseket érdemes jegyzetként használni a bemeneti fájlokban, hogy emlékeztessék Önt vagy bárki mást, aki a fájlt olvassa, mire szolgál az adott bemenet. A megjegyzésekkel a bemeneti fájl egy része ki is iktatható a beállítás idejére, ami gyorsítja a próbarendereléseket.

Egy bemeneti fájl általában egy stúdiódefinícióból áll, amely leírja a képméretet, az élsimítást és a nézőpontot, ezt követik a fényforrások, felületek és objektumok definíciói. Az objektumdefiníciók csak az objektum geometriájával foglalkoznak. Az objektum felületi jellemzői (szín, csillogás) az objektumot megelőző utolsó felületdefinícióból származnak.

Előfeldolgozó

A bemeneti fájlok írásának megkönnyítésére a Vivid értelmezője előfeldolgozóval is rendelkezik. Jelenleg az előfeldolgozó csak két funkciót támogat: makrók definiálását és include fájlok beillesztését. Az előfeldolgozó megvalósítása miatt a Vivid egy xyzzy.v nevű ideiglenes fájlt hoz létre a lemezen. (Nem említettem ezt már? Déjà vu.) Jelenleg a Vivid a használat után nem törli ezt a fájlt. Azért hagyom meg, mert néha hasznos belenézni, ha ki

akarjuk deríteni, mit művel az előfeldolgozó a bemeneti definícióval. Bármikor nyugodtan törölheti.

A makrókkal egy nevet rendelhetünk egy karaktersorozathoz. Amikor az értelmező a bemeneti fájlban meglátja a nevet, a folytatás előtt behelyettesíti a megfelelő karaktersorozatot. Egyszerű példa erre a color.vc fájl színdefiníciói. A color.vc-ben a fehér és a kék szín így van definiálva:

```
#define blue (0 0 1)
#define white (1 1 1)
```

Definiálás után a „white” szó bárhol használható, ahol egyébként (1 1 1)-et kellene begépelni. Például egy fehér csúcsfényes kék felület így hozható létre:

```
surface {
    diffuse blue
    shine 20 white
}
```

A makrónevek betűvel kezdődnek, és betűket, számokat, valamint aláhúzásjelet („_”) tartalmazhatnak. A makrónevek kis- és nagybetű-érzékenyek. Figyelje meg, hogy a példákban a színeket az rgb-értékek körüli zárójelekkel definiálom. Ez nem kötelező, de segít elkerülni az értelmező matematikai képességeiről szóló részben említett lehetséges problémákat. Egy makró definíciója a #undef kulcsszóval vonható vissza.

```
#undef blue
```

Ha ugyanazt a makrót többször definiálja, a régi értékek veremre kerülnek, és mindig a legújabb lesz érvényben. Ha ezután visszavonja a makrót, csak a legújabb törlődik, és az azt megelőző definíció lesz aktív. Ha egy sornál hosszabb makrót szeretne definiálni, a fordított perjelet („\”) kell folytatójelként használnia.

```
#define BLUE_PLASTIC  \
    surface {         \
        diffuse blue  \
        shine 20 white \
    }
```

Figyelje meg, hogy az utolsó sor után nincs fordított perjel. Definiálás után a BLUE_PLASTIC név bárhol használható a bemeneti fájlban, ahol egyébként a teljes felületdeklarációt kellene begépelni.

A másik előfeldolgozó-direktíva az #include. Ezzel más fájlok illeszthetők a bemeneti fájlba. Nézze meg a minta bemeneti fájlokat: szinte az első dolog bennük egy ilyen sor:

```
#include color.vc
```

Ennek hatására az értelmező beolvassa a color.vc fájlt, amely a bemeneti fájlokhoz használható szabványos színkészletet definiálja. Dan Farmer színszerkesztőjével nagyon egyszerűen hozhatók létre új színek ehhez a fájlhoz. Az include parancs objektumok jelenetbe illesztésére is hasznos. A transzformációs parancsokkal kombinálva lehetővé teszi, hogy az objektumokat külön fájlokban hozza létre, és a jelenet tetszőleges helyére, tetszőleges orientációval illessze be őket. Egy objektumból több példány is beilleszthető.

A stúdió

A jelenetfájl stúdió-struktúrája határoz meg mindent, ami nem fényforrás, objektum vagy vágás. Ide tartozik a végső kép felbontása, a kamera helye (nézőpont), a kamera iránya, a

háttérszín és különféle renderelési beállítások. Egyes beállításoknak alapértelmezett értékük van; ezt az értéket veszik fel, ha kimaradnak a stúdiódefinícióból. A stúdiódefiníció így néz ki:

```
studio {
    from x y z
    at x y z
    up x y z
    angle ang
    resolution i j
    start line
    stop line
    aspect asp
    projection mode

    ambient acolor
    background bcolor
    haze density

    antialias mode
    threshold dist
    jitter

    aperture size
    focal_length dist
    samples n

    no_shadows
    no_exp_trans
    caustics

    depth max_depth
    bunching n
}
```

from — A kamera helye.

at — A jelenet azon pontja, amely felé a kamera néz.

up — Felfelé mutató vektor, általában 0 0 1. Ezzel játszva a kép a kamera tengelye körül elforgatható.

angle — A látószög fokban.

resolution — A kép mérete pixelben: i pixel széles és j pixel magas. Parancssorból a -i kapcsolóval is megadható.

start, **stop** — Ezekkel a megadott sorszámoknál kezdhető, illetve állítható le a renderelés. Én új bemeneti fájlok tesztelésére és hibák behatárolására használom. Akinek hálózata van, ezzel feloszthatja a képet több számítógép közötti renderelésre. Amikor a részek elkészültek, a paste.exe-vel összeragaszthatók. Jelenleg probléma van a start és a folytatás (-r) kapcsoló együttes használatával – kerülje ezt a kombinációt.

aspect — A képernyő képaránya, vagyis annak a képernyőnek a szélesség/magasság aránya, amelyre a képeket rendereli. Úgy találtam, hogy az én NEC 3D-m képaránya kb. 4/3, az Amiga 1084-emé pedig kb. 1,2. A képernyő helyes képarányának meghatározásához mérje meg egy képernyőkép szélességét és magasságát, majd ossza el a szélességet a magassággal. A helyes képarány biztosítja, hogy a körök körnek, ne oválisnak látsszanak. Ne feledje: a képaránynak a megjelenített kép szélesség/magasság arányának kell lennie, a kép felbontásától függetlenül. A felbontással együtt ebből határozza meg a program a pixelek képarányát.

projection — Azt szabályozza, hogyan képeződik le a 3D világ a 2D képernyőre. Az alapértelmezés a „flat” (sík): ez a jól ismert és szeretett szabványos perspektivikus vetítés.

A „spherical” (gömbi) vetítés a halszemoptikához hasonló hatást ad; a dolgok elég furcsán nézhetnek ki. A „spherical” mód egyedi sajátossága, hogy a látószög 180 foknál nagyobb is lehet – próbáljon ki egyszer egy 360 fokos panorámát. Az „orthographic” (ortografikus) vetítési mód olyan képet ad, ahol minden szemsugár párhuzamos egymással. Ebben a módban az „angle” paraméternek nincs értelme, helyette a „width” (szélesség) paraméter szolgál, amely azt adja meg, milyen széles a képernyő a világ koordináta-rendszerében. Általában szerintem ez a mód haszontalan, de könnyű volt leprogramozni, úgyhogy miért ne? Az utolsó a „parallax” vetítési mód, amely a „flat” mód hárompontos perspektívája helyett kétpontos perspektívát ad. „Parallax” módban minden függőleges vonal függőleges marad a képernyőn. Kifejezetten építészeti látványtervekhez készült, de más esetekben is hasznos lehet. A „függőleges” irányt az up vektor határozza meg.

ambient — A képen mindenhol jelen lévő fény színe. A „valós világban”, amikor a fény szórt (diffúz) felületeket, például falakat ér, egy része visszaszóródik a helyiségbe. Ezért látunk az íróasztal alá akkor is, ha közvetlenül oda nem süt fény. A legtöbb sugárkövető, köztük a Vivid sem tudja kezelni ezt a diffúz kölcsönös visszaverődést. De nincs veszve minden: ennek imitálására a Vivid lehetővé teszi egy környezeti (ambient) fényérték beállítását. Ez olyan fényként viselkedik, amely egyszerre minden irányba világít, és nem vet árnyékot. Beltéri jelenetnél kb. .2 .2 .2 körüli érték jól működik. A kültéri jelenetek magasabb ambient értékkel kicsit valószerűbbek, a légkör fényszórása miatt. A legtöbb sugárkövetett képen az ambient érték 0 0 0, azaz fekete; ez adja a legélesebb kontrasztot, és hiperrealisztikus hatást kelt. Az ambient alapértéke 0 0 0.

background — Az a szín, amelyet a program akkor ad vissza, ha a sugár egyetlen objektumot sem talál el. Népszerű választás a black és a sky_blue. Ha köd (haze) van megadva, akkor ez a köd színe (lásd lent). A háttérszín alapértéke fekete. Alternatívaként a következő is megadható:

```
background {
    palette.map
    up x y z
}
```

Ez a palette.map fájl színeiből színátmenetes hátteret hoz létre. A fájl neve tetszőleges, de .map kiterjesztésűnek kell lennie, és formátuma megegyezik az img2gif palettafájljaiéval. A fájl első színe az up vektor irányában jelenik meg. Ha nincs up vektor megadva, a stúdió-struktúra up vektora érvényes.

haze — A köd vagy pára sűrűsége a jelenetben. Alapértéke 0, azaz nincs köd. A .5 érték azt jelenti, hogy a sugár által megtett minden egységnyi úton színének fele a háttérszínből adódik. A függvény exponenciális: ha a köd sűrűsége .5, akkor egy 1 egységnyit haladó sugár színe .5-ször az eltalált objektum színe és .5-ször a háttérszín. Egy 2 egységnyit haladó sugár színe .25-ször az objektum színe és .75-ször a háttérszín. A legtöbb képnél ez a paraméter figyelmen kívül hagyható, és maradhat a 0 alapérték.

antialias — Meghatározza, történjen-e élsimítás, és ha igen, milyen típusú. Parancssorból a -a kapcsolóval is szabályozható. Érvényes módok:

```
none     -- Pixelenként egy sugár, pontosan a közepén át. Az eredmény
            kockás, de viszonylag gyors. Ez az alapértelmezés.
quick    -- A kép részmintavételezéses közelítése. Az egyenletes színű
            területeken nyerhető a legtöbb gyorsulás. Ez a leggyorsabb mód,
            de az eredmény alig jó többre próbaképeknél. Legjobb esetben
            (pl. üres képnél) kb. 15-ször gyorsabb az előzőnél; általában
            3–5-szörös gyorsulást tapasztalok. Kiváló mód próbarenderekhez.
corners  -- Egy-egy sugarat lő a pixel minden sarkába, és átlagolja az
            eredményt. Mivel a sarkokon a szomszédos pixelek osztoznak,
            ez kb. pixelenként egy sugarat jelent. Majdnem olyan gyors,
            mint a none, de általában szebb. Gyakorlatilag ugyanaz, mintha
            simítószűrőt futtatnánk a képen.
adaptive -- A pixel sarkaiba lő sugarakat. Ha ezek egy bizonyos küszöbön
            belül vannak egymáshoz képest, a program továbblép a következő
            pixelre. Ha a küszöbnél jobban eltérnek, a pixelt négy
            alpixelre bontja, és újra mintavételez. Az alpixelek sarkait
            ismét a küszöbhöz hasonlítja, és ha még mindig túl messze
            vannak, újra felosztja őket. Ennek eredményeként az állandó vagy
            egyenletesen változó intenzitású területeken pixelenként csak
            egy sugár indul. Éleknél és más éles színátmeneteknél akár 25
            sugár átlaga is meghatározhatja a pixel színét. Az eredmény
            elég jó élsimítás, különösebb túlzott többletterhelés nélkül.
```

threshold — Az adaptív élsimítási mód küszöbértéke. Az alapérték 16, az érvényes tartomány 0..255. A paraméter a quick módra is hatással van. Általában az alacsonyabb értékek jobb eredményt adnak, de több időt vesznek igénybe.

jitter — Jelző, amely arra utasítja a rendszert, hogy minden kilőtt sugár irányához egy kis véletlenszerűséget adjon. Élsimítással kombinálva segít felbontani azokat a mintázatokat, amelyeket a kép szabályos rácson történő mintavételezése néha okoz. A szabályos mintázatú képek – például a horizontba vesző sakktáblák – profitálnak belőle a legtöbbet.

aperture — Opcionális paraméter, amellyel a sugárkövető valószerűbb kamerát modellezhet. Az alapértelmezett rekesznyílás 0, ami lyukkamerát modellez. 0-nál nagyobb rekesznyílásnál a fókusztávolságban (lásd lent) lévő objektumok élesek, míg a nézőponthoz közelebbi vagy távolabbi objektumok elmosódnak. Minél nagyobb a rekesznyílás, annál erősebb az elmosódás. Ez az opció jelentősen megnöveli a kép elkészítéséhez szükséges időt, mert a Vivid elosztott sugárkövetéssel (distributed ray tracing) modellezi a nem nulla rekesznyílású kamera hatását, így egy pixel színének kiszámításához szükséges sugarak száma nagyon megnő. Alapértelmezésben egy sugár helyett 8 sugarat lő ki, ha a rekesznyílás nagyobb nullánál. Ez az érték az alábbi „samples” paraméterrel szabályozható.

focal_length — A kamera és az élesen leképezett objektumok fókuszsíkja közötti távolság. Az aperture opcióval együtt használatos. A kamerától a fókusztávolsággal megegyező távolságra lévő objektumok élesek lesznek. Alapértéke a nézőpontot és a nézési irányt meghatározó „from” és „at” pontok távolsága.

samples — A nem nulla rekesznyílásnál kilőtt sugarak számát szabályozza. Alapértéke 8.

no_shadows — Kikapcsolja az összes árnyékszámítást. Az árnyékok kikapcsolásával nyert sebességnövekedés különösen hasznos egy új jelenet próbaképeinél. A -n parancssori kapcsolóval is szabályozható.

no_exp_trans — Csodálatosan intuitív név egy olyan jelzőhöz, amely kikapcsolja a sugarak exponenciális gyengülését az átlátszó objektumokon való áthaladáskor. Érthető? Hadd próbáljam újra. Normál esetben, amikor a Vivid egy sugarat egy átlátszó objektumon (üvegen) lő át, a sugár színét az üveg színe árnyalja, és ez az árnyalás attól függ, mekkora utat tesz meg a sugár az üvegben. Például ha van két kólásüveg-zöld üveglapunk, az egyik

1/4 hüvelyk, a másik 2 hüvelyk vastag, a vastagabbon áthaladó fény sötétebb lesz. A vastagság és az árnyalás mértéke közötti összefüggés exponenciális. Ez az egyoldalú üvegnél gondot okoz, mert amikor a Vivid az üveg vastagságát próbálja kiszámítani, a távolságot az üvegtől ahhoz a falhoz vagy padlóhoz méri, amelyet a sugár legközelebb eltalál. Így az ablakok általában nagyon sötétek lesznek. Ha a stúdió-struktúrában beállítja a no_exp_trans jelzőt, a Vivid az árnyalás kiszámításához csak a felület átlátszósági színét használja, és teljesen figyelmen kívül hagyja a sugár által megtett távolságot. Ez az árnyalás az árnyéksugarakra is hatással van.

caustics — Kísérleti jelző, amely bekapcsolja a Vivid „hamisított” kausztikáit. A kausztikák azok a szép fénymintázatok, amelyek akkor keletkeznek, amikor a fény átlátszó objektumon halad át. Kísérleteztem néhány módszerrel, hogy a hatás egy részét elérjem anélkül, hogy képenként napokat kelljen az igazi kiszámításával töltenem. A hatás elég finom, de úgy tűnik, néhány képet szebbé tesz. Joe Bob szerint érdemes kipróbálni.

depth — Korlátozza a sugárkövetés maximális rekurziós szintjét. 1-es mélységnél csak a szemsugarak követése történik. 2-es mélységnél az elsőszintű visszaverődések és fénytörések is. A legnagyobb megengedett érték 20, ez egyben az alapérték is. A -d parancssori kapcsolóval is módosítható.

bunching — A befoglaló dobozokból épített fa elágazási tényezőjét szabályozza. Nem igazán tudom, mi az optimális érték, de a 4-es alapérték a legtöbb esetben jól működik. Mindenesetre a kettőnél kisebb értékek garantáltan bajt okoznak. A nagyobb értékek hatására a befoglalódoboz-fa szintenként jobban elágazik, így laposabb lesz; a kisebbek az ellenkezőjét eredményezik. Azt tapasztalom, hogy a kísérletezés több időbe kerül, mint amennyit megtakarítok vele, ezért csak akkor használom, amikor közel járok a memória kifogyásához. Egy nagyobb érték, például 8 vagy 12, kevesebb összetett (befoglaló doboz) csomópontot hoz létre a fában, és memóriát takarít meg. Nyugodtan ne hagyja figyelmen kívül, attól való félelem nélkül, hogy lemarad valamiről.

Fényforrások

A fényforrásoknak négy fajtája van: pontszerű (point), irányított (directional), gömb alakú (spherical) és spotlámpa (spot). Ahogy a neve is mutatja, a pontszerű fényforrás a tér egyetlen pontját foglalja el. Van helyzete, színe, és olyan jellemzői, amelyek meghatározzák, mennyire csökken a fény intenzitása a távolsággal. Az irányított fényforrás végtelen távoli pontszerű fényforrásként viselkedik, intenzitása nem csökken a távolsággal. A gömb alakú fényforrásnak tényleges sugara van, és félárnyékos (lágy szélű) árnyékokat adhat; ez azonban jelentősen megnöveli a renderelési időt. A spotlámpák a széleken elhalványuló fénykúpot adnak. Ezek remekül alkalmasak a modell egyes területeinek kiemelésére. A legtöbb esetben ezek generálják a legkevesebb árnyéksugarat is, így gyorsabbak, mintha egyszerűen egy pontszerű fényforrást tennénk egy kúp belsejébe.

A pontszerű fényforrás definíciója:

```
light {
    type point
    falloff f       // alapérték: 0
    position x y z
    color r g b
}
```

A falloff paraméter határozza meg, hogyan csökken a fény intenzitása a távolsággal (dist). A valós világban a fényintenzitás 1/(dist*dist) szerint csökken (f=2). A Vivid azt is lehetővé teszi, hogy a fény 1/dist szerint csökkenjen (f=1), vagy egyáltalán ne csökkenjen (f=0). Miért használnánk bármi mást, mint f=2? Az egyik ok az egyszerűség. f=0 esetén a fény színét 1 1

1-re állíthatja, és tudhatja, hogy a megvilágított objektumok teljesen meg lesznek világítva, a fényforrástól való távolságuktól függetlenül. f=2 esetén ezt a távolságot figyelembe kell venni. Ha a megvilágítandó objektum 3 egységre van a fényforrástól, akkor az f=0-val azonos megvilágításhoz a színt 9 9 9-re kell állítani, azaz 3^2-re. f=1 esetén a szín 3 3 3 lenne. A valós világban a körülöttünk lévő fény nagy része nem közvetlenül a fényforrásból érkezik, hanem gyakran más objektumokról verődik vissza, mielőtt a minket érdeklő objektumhoz ér. Mivel a Vivid – a legtöbb sugárkövetőhöz hasonlóan – nem modellezi ezt az objektumok közötti diffúz visszaverődést, hasonló hatás érhető el, ha a fényintenzitás lineárisan csökken a távolsággal, azaz f=1. Az alapérték f=0.

Az irányított fényforrás definíciója:

```
light {
    type directional
    color r g b
    direction dx dy dz
}
```

vagy az irány helyett from és at pár is megadható:

```
light {
    type directional
    color r g b
    from x y z
    at x y z
}
```

Az irányvektor a fény haladási irányába mutat. Mivel a fényforrást végtelen távolinak tekintjük, nincs falloff paraméter. Ha nehezen érti, hogyan működik a direction paraméter, érdemes tudni, hogy a direction x y z ugyanaz, mint a center 0 0 0 at x y z.

A gömb alakú fényforrás definíciója:

```
light {
    type spherical
    position x y z
    radius r
    color r g b
    falloff f
    samples n
}
```

A gömb alakú fényforrások abban különböznek a pontszerűektől, hogy az általuk vetett árnyéknak félárnyéka van. Normál esetben, amikor egy sugár eltalál egy felületet, minden fényforrás felé egy árnyéksugár indul. Ha az árnyéksugár útközben bármilyen felületet eltalál, a fény takarva van, és a felület a takaró objektum árnyékában van. Gömb alakú fényforrásnál több árnyéksugár indul, mindegyik a fényforrás sugarán belüli egy-egy véletlen pont felé. Ha a fényt egy objektum félig takarja, az árnyéksugarak kb. fele elakad, fele eljut a fényforrásig. Az elakadt és nem elakadt árnyéksugarak arányából határozza meg a program az árnyék erősségét. Ahogy várható, a többlet árnyéksugarak sok időt adnak a rendereléshez. Egyes források kiterjedt fényforrásnak (extended light source) nevezik ezeket. Az egyszerre kilőtt árnyéksugarak számát a samples paraméter szabályozza, alapértéke 16.

A spotlámpa definíciója:

```
light {
    type spot
    position x y z
    direction dx dy dz
    min_angle angle1
    max_angle angle2
    color r g b
    falloff f
}
```

Az irányított fényforráshoz hasonlóan a direction paraméter itt is helyettesíthető az at x y z megadással, amely azt jelöli, hová világít a fény. A min_angle és max_angle a spotlámpa fénykúpjának alakját határozza meg. A fény tengelyétől min_angle szögön belül minden teljesen meg van világítva. Innen a fényintenzitás a max_angle eléréséig csökken. Például ha 30 fok széles, éles szélű fénykúpot szeretne, a min_angle-t és a max_angle-t is 30-ra kell állítani. Ugyanekkora, de a széleken elhalványuló fényhez a max_angle legyen 30, a min_angle pedig 0.

Minden fényforrásnak lehet még néhány további paramétere: no_shadows és no_spec. Ahogy valószínűleg kitalálta, ezekkel fényforrásonként kikapcsolhatók az árnyékok és a csúcsfények. Egy kis intenzitású (.2 .2 .2) irányított fényforrással együtt használva ez gyakran jó alternatívája a globális ambient fénynek. A globális ambient fény hajlamos laposnak mutatni az objektumokat, míg ez a technika finom árnyalást ad, az „igazi” fényforrások árnyékai és csúcsfényei nélkül.

Fontos megjegyezni, hogy a fényforrás akkor sem jelenik meg objektumként, ha a látótéren belül van. Ha azt szeretné, hogy a fényforrások látszódjanak, egy gömbbel átlátszó burkot tehet köréjük. Példa:

```
// Pontszerű fényforrás a 2 3 4 helyen, amely 1 sugarú fényként
// jelenik meg a jelenetben.

light {
    center 2 3 4
    type point
    color white
}

// üvegburok

surface {
    ambient white       // ugyanaz a szín, mint a fényé
    transparent white   // teljesen átlátszó
}
sphere {
    center 2 3 4
    radius 1
}
```

A gömbök és felületek részleteiért ugorjon előre.

Felületek

A felület-struktúrákkal a renderelt objektumok felületi jellemzői adhatók meg, például szín, tükrözőképesség és textúra. Amikor a bemeneti fájlban egy felület definiálva van, az az utána következő összes primitívre érvényes, amíg új felületet nem definiálunk. Így több objektum is megadható anélkül, hogy a felületi jellemzőket ismételni kellene. Egy egyszerű felület-struktúra így néz ki:

```
surface {
    diffuse r g b        // alapérték: 0 0 0 (fekete)
    ambient r g b        // alapérték: 0 0 0
    specular r g b       // alapérték: 0 0 0
    shine pow            // alapérték: 0
    transparent r g b    // alapérték: 0 0 0
    ior num              // alapérték: 1.0
    fuzz magnitude       // alapérték: 0.0
    no_antialias         // élsimítás kikapcsolása
}
```

A felület összes összetevőjét figyelembe veszi a sugárkövető, amikor meghatározza egy objektum színét a jelenetben. A diffúz szín az objektum tényleges színe, ahogy teljes fehér fényben látszik. A 0 0 0 érték fekete, az 1 1 1 fehér objektumot jelent. Ennek az összetevőnek a fényessége a felület adott pontjára eső fény mennyiségétől függ. Az ambient tagot néha önvilágító összetevőnek is nevezik: ez az a szín, amilyennek az objektum teljesen sötét jelenetben látszik. A specular összetevő a felület tükrözőképességét adja meg; az 1 1 1 érték tükörszerű visszaverődést eredményez. A shine érték határozza meg, mekkora legyen a csúcsfény-folt a felületen. Az alacsony értékek (1..10) nagy, lágy szélű csúcsfényt, a magas értékek (1000 vagy több) kicsi, éles foltot adnak. Hagyományosan a folt fényessége és színe egyenesen arányos a specular összetevővel. A gond az, hogy néha jó lenne egy kék objektum csúcsfényekkel, a visszavert sugarak követésének többletterhe nélkül. Ezért a Vivid a csúcsfények megadásának egy második formáját is lehetővé teszi:

```
shine pow r g b
```

Ebben az esetben a megadott színt használja a felület specular összetevője helyett. A transparent összetevővel adható meg, mennyire átlátszó a felület. Az 1 1 1 érték üvegszerű, mert minden színt átenged, míg az 1 0 0 érték vörös üveghez hasonló felületet ad, mivel csak a vörös fényt engedi át. A .9 .9 .9 átlátszósági összetevőjű felület részben átlátszónak tűnik; az átengedett fény mennyisége attól függ, milyen vastag az objektum, amelyen a fény áthalad. A törésmutató (ior) határozza meg, mennyire törik meg a sugár, amikor az átlátszó felületbe lép. A valóságban ez a felület relatív sűrűségével függ össze. Üveg szimulálásához kb. 1,1–1,3 közötti értékek működnek a legjobban. A gyémánt törésmutatója 2,6. A fuzz egy módja annak, hogy a színmeghatározáskor véletlen zajt adjunk az objektum felületi normálisához. Mivel az objektum diffúz színét befolyásolja a fény beesési szöge, ez a véletlenszerűsítés egyfajta durva textúrát adhat az objektumnak. Tükröző vagy átlátszó felületeken a matt (homokfúvott) üveghez hasonló hatást kelt. Általában a kis fuzz-értékek (.01 és .3 között) működnek a legjobban.

A no_antialias jelző arra utasítja az adaptív élsimítást, hogy az adott felületen gyakorlatilag kapcsoljon ki. Ezt általában nem érdemes használni, csak néhány speciális esetben. A paraméter eredeti oka az volt, hogy az elmosódott (fuzzy) felületek miatt az adaptív élsimítás rengeteg sugarat lőhet ki, és jelentősen lelassíthatja a képgenerálást. Ha a felületdefinícióhoz hozzáadja a no_antialias jelzőt, az objektumok élein továbbra is megkapja az adaptív élsimítás előnyeit, de elkerüli a nagy, elmosódott felületek okozta lassulást. Vegye figyelembe azonban, hogy ez megváltoztatja a felület megjelenését. Az opció használatakor próbálja meg felére csökkenteni a fuzz mértékét, hogy megmaradjon a felület színváltozatossága.

A gépelni nem tudók kedvéért egyes kulcsszavak rövidíthetők: surf, diff, amb, spec és trans. Bízom benne, hogy ki tudja találni, mi mihez tartozik.

Lássunk néhány példát:

```
// egyszerű vörös felület
surface {
    diff 1 0 0
}

// önvilágító kék
surface {
    ambient 0 0 1
}

// tükör csúcsfényekkel
surface {
    spec 1 1 1
    shine 100
}

// üveg némi tükröződéssel
surface {
    spec .3 .3 .3
    shine 30
    trans .7 .7 .7
    ior 1.2
}
```

Általános ökölszabályként az amb+diff+spec+trans összege legyen kisebb vagy egyenlő, mint 1 1 1. Persze mivel a saját univerzumunkat definiáljuk, bármi lehetséges.

Bump mapping (felületi egyenetlenség)

A bump mapping egy módszer arra, hogy a felületnek textúrát adjunk, amely fodrok vagy dudorok hozzáadásával fokozza a felület valószerűségét. A felület valójában nem torzul, csak a normálisa. Így egy egyszerű felület is nagyon bonyolultnak tűnhet. A bump definíciók az egyszerű felületdefiníciókon belül szerepelnek, azaz:

```
surface {
    diffuse red
    bump {
        ...
    }
}
```

Egy hullám típusú bump map minta:

```
bump {
    wave {
        center 1 2 3
        wavelength 2.0
        amplitude 0.2
        damping 0.9     // alapérték: 1.0
        phase 0.0       // alapérték: 0.0
    }
}
```

A center a hullám forrását határozza meg. A wavelength a hullám két hegye közötti távolság. Az amplitude a felületi normális maximális kitérítésének mértéke; az 1 alatti értékek egyértelműen a legjobbak. A damping paraméter azt adja meg, mennyire csökken az amplitúdó a távolsággal. A példában az amplitúdó a forrástól mért minden hullámhossznyi távolságon 10%-kal csökken. A phase egy 0 és 1 közötti szám, amely a hullám fázisának kezdeti eltolását határozza meg. Animációkban ezzel valószerűen mozgó hullám készíthető, ha képkockánként kis mértékben növeljük a fázist. A bump struktúrán belül több hullám is definiálható. Három-négy különböző helyen lévő, eltérő hullámhosszú és amplitúdójú hullámforrással nagyon valószerű, fodrozódó felület hozható létre.

A felület normálisa turbulenciával is megzavarható. A turbulencia definíciója:

```
bump {
    turbulence {
        scale 1 1 1
        offset 0 0 0
        amplitude .5
        terms 4
    }
}
```

A turbulenciafüggvény a sugár metszéspontjának helyéből egy ± amplitude tartományba eső véletlen számot ad vissza. A scale és offset tényezőket a program a turbulenciafüggvény meghívása előtt alkalmazza az xyz helyre. A terms paraméterrel fraktálszerű felület építhető: ha terms>1, a turbulenciafüggvény többször összegződik, és az összeg minden további tagjánál a skálázás duplázódik, az amplitúdó feleződik. Ez adja a fraktálokra jellemző, különböző szintű önhasonlóságot. A vivid1.v mintafájl ezt a funkciót és nem szimmetrikus skálázást használ a nagy gömb „szálcsiszolt króm” megjelenéséhez. A turbulencia- és hullámdefiníciók egy bump definíción belül együtt is szerepelhetnek.

Textúrázott felületek

A Vivid a renderelt felületek valószerűségének fokozására térbeli (solid) textúrázást is lehetővé tesz. A textúrázott felületek valójában két felületből állnak, valamint annak a meghatározásából, hogy melyik hol jelenik meg az objektumon, és hogyan keveredik a kettő. A felületek háromféle mintázatban „rétegezhetők”: checker (sakktábla), spherical (gömbi) vagy noise (zaj). A checker mintázat térbeli sakktáblát ad. A kockák méretét a scale tényező szabályozza. Ha valamelyik scale paraméter 0, akkor a mintázat az adott tengely mentén nem változik. Példa: egyszerű, 2 egység oldalú fekete-fehér sakktábla, amely a z tengely mentén végtelenül kiterjed.

```
surface {
    texture {
        pattern checker
        scale 2 2 0
        offset 0 0 0    // alapérték
        fuzz 0          // alapérték
        blend 0         // alapérték
        surface { diff black }
        surface { diff white }
    }
}
```

A 2 0 0 skála olyan mintázatot adna, amely x irányban 2 egységenként változik, de y és z irányban folytonos. Ez az x tengely mentén egymásra rakott, 2 egység vastag anyagrétegeknek felel meg. A spherical mintázat egymást váltó felületekből álló koncentrikus rétegeket ad. Ha valamelyik scale paraméter 0, koncentrikus hengerek keletkeznek, amelyek tengelye a nullázott tengely mentén fut. Ez fatextúrákhoz hasznos. A spherical mintázathoz két sugárdefiníció is szükséges a rétegekhez: az első sugár az első felülethez tartozik, és így tovább.

```
surface {
    texture {
        pattern spherical
        scale 2 2 0
        radius 1
        radius 2
        surface { diff black }
        surface { diff white }
    }
}
```

A noise mintázat a zajfüggvény kimenetét közvetlenül a két felület közötti választásra használja. Ez gránitszerű textúrákhoz hasznos. Az x, y és z irányban eltérő skálázással csíkos megjelenésű felület kapható. Barna és drapp felületekkel finom faerezetes hatást is elértem vele.

```
surface {
    texture {
        pattern noise
        terms 4
        scale x y z
        surface { diff white }
        surface { diff black }
    }
}
```

A fuzz és blend paraméterekkel a két felület közötti határ lágyítható. Értékük 0 és 1 között lehet. A blend paraméter sima átmenetet hoz létre a felületek között; értéke az átmeneti sáv szélességét határozza meg. A fuzz paraméter az értékével arányos zajt ad a vizsgált ponthoz, ami durva, pöttyös átmenetet eredményez a felületek között.

A bump mapről szóló részben említett turbulenciafüggvény textúrázott felületekre is alkalmazható. A paraméterek változtatásával a hatás a mintázat enyhe megzavarásától a márványos megjelenésen át a túlzott turbulenciáig terjedhet. Egy egyszerű példát alább mutatok be.

A Vivid értelmezőjének működése miatt, ha egy textúrázott felületet bump mappel is el szeretne látni, a bump definíciónak a felület-struktúrában a textúradefiníció előtt kell szerepelnie. Figyelje meg azt is, hogy a textúradefinícióban megadott felületeknek nem kell egyszerű felületeknek lenniük – maguk is lehetnek textúrázottak. Az alábbi definíció egy 10 egység oldalú kockákból álló sakktábla-felületet ad; a kockák fele fekete, a másik fele vörös-fehér márványos.

```
surface {
    texture {
        pattern checker
        scale 10 10 0
        surface { diff black }
        surface {
            texture {
                pattern checker
                scale 1 0 0
                blend 0.7
                turbulence {
                    amplitude 3
                    terms 4
                }
                surface { diff white }
                surface { diff red }
            }
        }
    }
}
```

Mandelbrot

A Vivid Mandelbrot-mintázatot is ismer, ahogy a vivid1.v mintafájl bemutatja. A Mandelbrot-mintázat egy este viccből került a Vivid-be. Jelenleg a megadási formátuma borzalmas. A scale érték első két paramétere a halmaz x és y skálázását, a harmadik a maximális iterációszámot határozza meg. Az iterációs számítás eredményét a program elosztja a maximális iterációszámmal, és ez adja meg az adott pontban a felületi paraméterek egymáshoz viszonyított arányát.

Rendben, a működés a következő. A tényleges metszéspontot (x y z) a program a scale és offset értékekkel eltolja, mielőtt a Mandelbrot-számításba kerül.

```
new_x = x * scale_x + offset
```

Ahogy mondtam, ezt sikerült a lehető legfordítottabbra megcsinálnom úgy, hogy még működjön. Mivel a pontot megszorozzuk a skálával ahelyett, hogy elosztanánk, a mintázat annál nagyobb, minél kisebb a skálaérték. Normál esetben a Mandelbrot-halmaz az x tengely mentén (nagyjából) -2-től +1-ig terjed. Ha azt szeretné, hogy -20-tól +10-ig terjedjen, az x és y tengelyen 0,1-gyel kell skáláznia. Ostobaság? Igen. Mivel pedig az offset a szorzás után adódik hozzá, az a Mandelbrot-koordináta-rendszerben hat, nem a világ koordináta-rendszerében. A fenti példát folytatva, ha a halmaz középpontját a 300, 100 világkoordinátákra szeretné helyezni, 30 10 offsetet kell megadnia.

Makrókkal szerintem a szemét nagy része kiküszöbölhető:

```
#define ITERATION_LIMIT (64)
#define SCALE (10)
#define X_OFFSET (300)
#define Y_OFFSET (100)

surf {
    texture {
        pattern mandelbrot
        scale (1/SCALE) (1/SCALE) ITERATION_LIMIT
        offset (X_OFFSET/SCALE) (Y_OFFSET/SCALE) 0
        // stb...
    }
}
```

Most már csak a define-okat kell módosítania, és a dolgok sokkal inkább úgy fognak működni, ahogy várja.

Amint kitalálom, hogyan lehet ezt értelmesen megoldani, a Mandelbrot- és Julia-halmazok a Vivid igazi részévé válnak. (Igen, tudom, ezt múltkor is mondtam, de most is igaz.)

Gömb

A gömb a Vivid által támogatott legegyszerűbb primitív, és általában ezzel végezhető el a leggyorsabban a metszésvizsgálat. A gömb formátuma:

```
sphere {
    center x y z
    radius r
}
```

ahol x y z a gömb középpontjának helye a térben, r pedig a gömb sugara. Példaként két üveggömb definíciója, az egyik üreges, a másik tömör.

```
// üvegfelület
surface { trans 1 1 1 shine 200 1 1 1 ior 1.2 }

// tömör gömb
sphere { center 1 0 0 radius .9 }

// üreges gömb
sphere { center -1 0 0 radius .9 }   // külső felület
sphere { center -1 0 0 radius .8 }   // belső felület
```

Csak a móka kedvéért a Vivid elmosódott (fuzzy) gömböket is ismer. Ezek azért tűnnek elmosódottnak, mert nincs rögzített sugaruk. Elmosódott gömb létrehozásához definiáljon egy normál gömböt, és adjon hozzá egy fuzz paramétert. Ez határozza meg, mennyivel lehet véletlenszerűen nagyobb a sugár. Minden alkalommal, amikor a sugárkövető metszésvizsgálatot végez az elmosódott gömbbel, a vizsgálathoz használt sugarat véletlenszerűen választja a radius és radius+fuzz közötti tartományból.

```
// elmosódott gömb 0,5 és 2 közötti sugárral
sphere {
    center 0 1 2
    radius .5
    fuzz 1.5
}
```

Gyűrű

A ring (gyűrű) primitív alátétként vagy korongként is leírható. A gyűrű definíciója egy helyből, egy felületi normálisból, valamint egy minimális és egy maximális sugárból áll. A minimális sugár lehet nulla is, ekkor középső lyuk nélküli korong keletkezik. Mivel a gyűrű metszésvizsgálata gyorsabb, mint a sokszögé, a gyűrű jó választás talajsíknak a sugárkövetett objektumok alá. A gyűrűdefiníció formátuma:

```
ring {
    center x y z
    normal a b c
    min_radius r0
    max_radius r1
}
```

Az a b c felületi normálisnak nem kell normáltnak lennie. Ha egyszerűen középső lyuk nélküli korongot szeretne, a min/max sugárdefiníciók egyetlen sugárdefinícióval helyettesíthetők:

```
ring {
    center x y z
    normal a b c
    radius r
}
```

Hoppá, most esett le, hogy nem mindenki tudja, mi az a felületi normális. A felületi normális egy felületre merőleges vektor, azaz olyan, amely egyenesen kifelé mutat a felületből. Például a szobája padlójának felületi normálisa egy egyenesen felfelé, a levegőbe mutató vektor lenne. A monitor felületi normálisa az a vektor, amely egyenesen kifelé mutat, az Ön két szeme közé. Elég egyszerű? Rendben, a normált vektor pedig olyan vektor, amelyet úgy skáláztunk, hogy hossza pontosan 1 legyen. Ez általában úgy történik, hogy kiszámítjuk a vektor hosszát, majd a vektor minden összetevőjét elosztjuk vele. A 0 hosszúságú vektorok végtelen sok bajt okoznak.

Sokszög

A sokszögeknek tetszőleges számú csúcsuk lehet (na jó, legalább három kell). A csúcsoknak mind ugyanabban a síkban kell lenniük, különben furcsa lesz az eredmény. A csúcsok sorrendje lehet az óramutató járásával megegyező vagy ellentétes.

```
polygon {
    points 4
    vertex  1  1 0
    vertex  1 -1 0
    vertex -1 -1 0
    vertex -1  1 0
}
```

Ez egy 2 egység oldalú, az origó középpontú négyzet alakú sokszöget hoz létre, amelynek felületi normálisa 0 0 1.

Háromszögfolt

A háromszögfolt (triangular patch) bonyolult alakú, simának látszó objektumok építésére hasznos. A foltot három csúcs és csúcsonként egy-egy explicit felületi normális határozza meg. Általában a foltok kézi bevitele valószínűleg túl fárasztó ahhoz, hogy igazán hasznos legyen, de számítógéppel generált bemeneti fájloknál az eredmény megéri a többlet programozási munkát.

```
patch {
    vertex 1 0 0    normal .1 0 1
    vertex 0 1 1    normal 0 .1 1
    vertex 0 0 .5   normal -.1 -.1 1
}
```

Kúp

A Vivid által megvalósított kúpok valójában csonkakúpok: az alapjuknál és a csúcsuknál is van sugaruk. Ha a két sugár egyenlő, a kúp elég jól utánozza a hengert. Hegyes kúphoz az egyik sugárnak 0-t adjon meg.

```
cone {
    base 1 1 1  base_radius 4
    apex 0 0 5  apex_radius 1
}
```

A gyűrűk jól használhatók a kúpok végeinek lezárására. A gyűrű helyzete és normálisa még ferde szögű kúpnál is könnyen kiszámítható. A kúp csúcsoldali végének lezárásához a gyűrű középpontja egyenlő az apex-szel, a gyűrű sugara az apex_radius-szal, normálisa pedig az apex-base különbséggel. A fenti példát használva a kúp csúcsoldali végét lezáró gyűrű definíciója:

```
ring {
    center 0 0 5
    radius 1
    normal -1 -1 4
}
```

Transzformációk

A Vivid transzformációs parancsaival az objektumok mozgathatók és skálázhatók. A transzformációs parancsok az őket követő összes primitívre érvényesek, amíg le nem „pop-oljuk” őket a transzformációs veremről. A transzformációs parancs formátuma:

```
transform {
    scale s
    rotate x y z
    translate dx dy dz
}
```

A scale az objektumok méretét változtatja meg. A scale vektorként is megadható, így tengelyenként eltérően skálázott objektumok hozhatók létre. Sajnos ez nem minden primitívvel működik: csak a sokszögek és foltok skálázhatók nem egyenletesen. A rotate parancs az objektumot minden tengely körül a megadott fokkal elforgatja. A translate parancs eltolja az objektumot. Bármelyik elhagyható vagy többször is használható. Tetszőleges sorrendben is megadhatók, és a megadás sorrendjében hatnak az objektumokra. Nagyon fontos, hogy a sorrend helyes legyen. Egy objektum, amelyet a Z tengely körül 90 fokkal elforgatunk, majd az X tengely mentén 10 egységgel eltolunk, a 10 0 0 pontba kerül 90 fokos elfordulással, míg ha a műveleteket fordított sorrendben alkalmazzuk, a 0 10 0 pontba kerül. Néha segít, ha először valódi tárgyakkal játszadozva végiggondolja a transzformációkat. Ne feledje, hogy minden forgatás a tengelyek körül történik, nem feltétlenül az objektum középpontja körül. Ezt új objektumok építésekor is érdemes szem előtt tartani: a 0 0 0 pont legyen az objektum „természetes” forgási középpontja. Ez nagyban megkönnyíti a jelenetek építését az objektumokból. Például egy autómodell természetes „középpontja” a talaj szintjén, az autó közepén lenne. Így az autó elég könnyen elhelyezhető a jelenetben, ahová szeretné.

Egy transzformáció a transzformációs veremről a „transform_pop” paranccsal távolítható el. Néha egymásba szeretné ágyazni a transzformációs parancsokat. Ez olyan többrészes objektumok létrehozásához hasznos, amelyek részei egymáshoz képest mozognak, de egységként is mozgatni kell őket. Tegyük fel például, hogy egy forgatható tornyú harckocsimodellt szeretne készíteni. Legyen a harckocsi teste a tank.vo, a torony a turret.vo fájlban. A harckocsi jelenetbe helyezéséhez a bemeneti fájl így nézne ki:

```
#define TURRET_ANGLE (30)        // a torony elforgatása

transform { translate x y z }    // az egész harckocsi mozgatása
#include tank.vo                 // a test geometriája
transform { rotate 0 0 TURRET_ANGLE }
#include turret.vo               // a torony geometriája

transform_pop                    // a transzformációs verem rendbetétele
transform_pop
```

Ezzel a technikával viszonylag könnyen építhetők és helyezhetők el bonyolult modellek. Jelenleg a transzformációk használatának van egy nagy hátránya: a felületi textúrák nem mozognak együtt az objektummal. Ez egyedi képeknél nem túl nagy gond, de a textúrázott objektumok animálását nagyon csúnyává teszi. Igen, dolgozom rajta.

Vágás (clipping)

A primitívek „vághatók” is, így bonyolultabb alakzatok hozhatók létre. A vágás lényegében úgy működik, hogy a primitívet egy vágófelülettel együtt definiáljuk, amely levágja a primitív egy részét. Például egy gömb egy síkkal elvágva félgömböt ad, vagy egy kúppal lyuk vágható egy másik kúpon keresztül. Háromféle vágófelület van: sík, gömb és kúp. A vágásokat a primitív definícióján belül kell megadni. Primitívenként több vágás is lehet. A vágások a primitívükkel együtt transzformálódnak.

A vágósíkot egy pont és egy normális határozza meg:

```
clip {
    center x y z
    normal x y z
}
```

A primitív síknak a normális felőli oldalán lévő része megmarad, míg a másik oldalon lévő rész eltűnik a Ronald Reagan Emlékkönyvtárban. (Azt hiheti, hogy hazudok, de sosem fogja igazán *tudni*, amíg el nem megy megnézni.) Például egy origó középpontú, 1 sugarú félgömb így néz ki:

```
sphere {
    center 0 0 0  radius 1
    clip {
        center 0 0 0  normal 0 0 1
    }
}
```

Figyelje meg, hogy a vágás normálisa felfelé mutat; így a gömb felső felét kapja. Ha a normálist 0 0 -1-re változtatja, az alsó felét kapja.

A vágógömbök definíciója:

```
clip {
    center x y z
    radius r
    inside  vagy  outside
}
```

Vágógömbnél választhat, hogy a primitívnek a gömbön belüli vagy kívüli része maradjon meg. Talán már kitalálta, de ezért vannak az inside és outside kulcsszavak.

A vágókúpok így néznek ki:

```
clip {
    apex x y z  apex_radius r
    base x y z  base_radius r
    inside  vagy  outside
}
```

A kúp primitívhez hasonlóan itt is megadható egyetlen sugár, így hengert kapunk.

Néha ugyanazokat a vágásokat egy primitívcsoportra szeretné alkalmazni. Ehhez a global_clip kulcsszóval globális vágásokat definiálhat:

```
global_clip {
    clip { ... }
    clip { ... }
    clip { ... }
}
```

A clip_pop kulcsszó hatására az előző vágáscsoport lekerül a veremről, hasonlóan ahhoz, ahogy a transform_pop a transzformációkkal teszi.

Segédprogramok

img2gif.exe

Amikor a Vivid elkészült egy képpel, valahogy meg kell nézni az eredményt. Itt jön képbe az img2gif. Az img2gif a Vivid által kiírt nyers img fájlt gif fájllá alakítja. Az így kapott gif fájl kedvenc képnézegetőjével megtekinthető. Az én kedvencem Brad Montgomery vpic-je – keresse a helyi BBS-en. Végszükség esetén akár a fractint is használható gifek megtekintésére.

Az img2gif futtatásakor két fő döntést kell meghoznia: a) hogyan válassza ki a program a palettát, és b) alkalmazzon-e a program ditheringet (raszteres színkeverést) a kimeneten. A paletta kiválasztása nem mindig egyszerű feladat. A szabványos VGA a rendelkezésre álló 262 144 színből csak 256-ot enged egyszerre megjeleníteni. Egy „átlagos” alacsony felbontású képen kb. 7000 szín van, amelyet 256 színű palettával kell ábrázolnunk. Egy nagyobb felbontású képen akár több tízezer ábrázolandó szín is lehet. Az img2gif jelenleg a következő algoritmusokat valósítja meg.

Palettafájl – A paletta .map fájlból is beolvasható. A formátum megegyezik a fractint által létrehozott és olvasott .map fájlokéval. Ez az opció hasznos, ha animációhoz több, azonos palettájú gifet akar készíteni.

Népszerűség (popularity) – A népszerűségi algoritmus úgy választja ki a palettát, hogy azt a 256 színt veszi, amely a legtöbb pixelt képviseli a képen. A kép többi színét legkisebb távolság alapján képezi le a palettára. A népszerűségi algoritmus általában a kevesebb színt tartalmazó képeknél működik a legjobban. Tipikus problémája, hogy a kis csúcsfények színe nem lesz helyes: a csúcsfények általában csak néhány pixelt fednek le, így színük nincs elég nagy arányban jelen a képen ahhoz, hogy az algoritmus kiválassza. A probléma enyhítésére az img2gif a színkocka csúcsait (fehér, vörös, zöld, kék, cián, bíbor, sárga és fekete) kényszeríti az első nyolc palettabejegyzésbe. Mivel a legtöbb csúcsfény fehér, ez nagyban csökkenti a nem kívánt képhibákat.

Median cut – A median cut algoritmus kiegyensúlyozottabb színkészletet próbál választani a kép ábrázolásához. Az alapötlet az, hogy olyan palettát válasszon, amelyben minden bejegyzés nagyjából ugyanannyi pixelt képvisel a képen. Ez segít helyesen színezni azokat a csúcsfényeket, amelyeket a tisztán népszerűségi algoritmus kihagyna. Az img2gif azt is lehetővé teszi, hogy korlátozza az egyetlen szín által képviselt pixelek számát. Ez gyakorlatilag megnöveli a csúcsfényekben és a kép más kis területein lévő színek fontosságát. Ha ez a korlát egy, a kép minden színe azonos súlyt kap, függetlenül attól, hány pixelt fed le.

Rögzített paletta – A rögzített paletta opció a kép tartalma alapján történő választás helyett előre meghatározott palettát használ. Ennek előnye, hogy sokkal gyorsabb. Népszerűség vagy median cut szerinti választásnál az img2gif-nek először fa szerkezetet kell építenie a memóriában, amely a kép összes színét és az egyes színek által képviselt pixelek számát tartalmazza. A paletta kiválasztása után a fában lévő színeket le kell képezni a paletta színeire. Ez elég sokat lassíthat az img2gif-en. Rögzített palettánál a kép színei egyszerű

egyenletekkel közvetlenül a paletta színeire képeződnek le, így nincs szükség a fa szerkezetre és a kép színeinek egy tetszőleges palettára történő költséges újraleképezésére. Ráadásul az img2gif jelenlegi verziója csak kb. 57 000 színt tartalmazó fát tud kezelni; ezen túl az én klónomból kifogy a RAM. Az img2gif által támogatott rögzített paletták:

```
0 : Szürkeárnyalatos kép 256 szürke tónussal. A VGA korlátai miatt
    a legtöbb PC-n 64 árnyalatként jelenik meg.

1 : A színkockát 8 vörös, 8 zöld és 4 kék árnyalatra osztja. Általában
    elég rossz képeket ad, de nosztalgiából megtartottam. Azért a kék
    kapta a kisebb felbontást, mert az emberi szem általában sokkal
    kevésbé érzékeny a kék tartományban, és nehezebben fókuszál az
    élekre, így a színfelbontás csökkenése kevésbé feltűnő.

2 : A színkockát 6 vörös, 7 zöld és 6 kék árnyalatra osztja. Ez az
    opció adja a legjobb egyensúlyt a sebesség és a jó színvisszaadás
    között. Azt tapasztaltam, hogy a sok színt tartalmazó képekkel
    működik a legjobban. A dithering hozzáadása általában segít.
```

A dithering az a folyamat, amelynek során különböző színű pixeleket úgy helyezünk egymás mellé, hogy több szín illúzióját keltsék. Hétköznapi példa erre a televízió. Dugja az orrát a képcsőhöz, és vegye észre, hogy a kép apró, változó intenzitású vörös, zöld és kék pontokból áll. A tévé valójában csak ezeket a színeket állítja elő, de a szeme ezekből rakja össze a spektrum többi részét. Na jó, legalábbis az NTSC szerinti spektrumot.

A Floyd–Steinberg dithering az eredeti hibaszétosztásos (error diffusion) dithering algoritmus. A hibaszétosztásos dithering úgy próbálja kompenzálni a korlátozott palettát, hogy a kép bármely területén a hibák összege nulla legyen, még ha az egyes pixelek színe tartalmaz is némi hibát. Ahogy az img2gif végighalad a képen, minden pixelhez a legközelebbi színt választja. Ez a szín általában nem pontos. A kiválasztott palettaszín és a valódi szín közötti különbséget a program hozzáadja a még nem feldolgozott szomszédos pixelekhez. A Floyd–Steinberg dithering előnye a „véletlenszerű” megjelenés: általában nem hoz létre észrevehető mintázatokat a kimeneti képen.

A rendezett (ordered) dithering úgy működik, hogy minden pixelhez a helyétől függő rögzített értéket ad. Ezeknek a hozzáadott értékeknek az összege a kép egy területén nulla. Mivel a rendezett dithering nem veszi figyelembe a szomszédos pixelek értékét, rögzített mintázatokat hoz létre a képeken, amelyek bizonyos esetekben zavaróak lehetnek. A rendezett dithering rögzített palettával kombinálva előnyös, ha a Vivid-del animáció egyes képkockáit készíti. Animáció közben a kép háttér- (nem mozgó) területei állandóak maradnak, míg Floyd–Steinberg ditheringnél a mintázatok folyamatosan vándorolnak a képernyőn mozgó objektumok körül és alatt.

A véletlen zaj opcióval fehér zaj adható a képhez. Bizonyos esetekben ez ugyanannyit segíthet, mint a dithering. Bármelyik dithering opcióval együtt is használható. A zaj tartományának növelésével érdekes hatások érhetők el: a nagy zajjal átalakított képek kifejezetten szemcsés megjelenést kapnak. Érdekes, de nem túl hasznos. A zaj alapértelmezett tartománya ±8 a 0..255 skálán. Ezt az értéket kizárólag azért választottam, mert a legtöbb esetben jól nézett ki. A véletlen zaj előnye, hogy képről képre állandó, így egy képsorozat sima területei animáció közben nem vibrálnak.

Na jó, de hogyan futtatom az img2gif-et? Így:

```
img2gif [-m #] [-f #] [-p palfile] [-d] [-o] [-r [#]] file
```

Az alapértelmezett palettaválasztó algoritmus a népszerűségi módszer.

```
[-m #]        median cut kiválasztása; minden szín legfeljebb # pixelt
              képviselhet. # alapértéke 64K.
[-f #]        # számú rögzített paletta kiválasztása. # alapértéke 0
              (szürkeárnyalatos).
[-p palfile]  a palettát a palfile.map fájlból olvassa be. Ha ez a fájl
              nem létezik, a palettát – a többi beállított kapcsolótól
              függően – median cut, népszerűség vagy rögzített paletta
              alapján választja ki, majd a kiválasztott palettát a
              palfile.map fájlba írja.
[-d]          Floyd–Steinberg dithering.
[-o]          rendezett dithering.
[-r #]        ± # véletlen zaj. # alapértéke 8.
```

Például:

```
img2gif -f 2 -d test
```

a test.img fájlt test.gif fájllá alakítja a 2-es rögzített palettával és Floyd–Steinberg ditheringgel.

Feltéve, hogy a test.map nem létezik:

```
img2gif -m -p test test1
img2gif -m -p test test2
img2gif -m -p test test3
img2gif -m -p test test4
```

a test1-et median cut palettaválasztással alakítja át, majd a palettát a test.map fájlba menti. A test2, test3 és test4 átalakítása ezután a test.map palettájával történik.

Általában a legjobb eredményt a -m -d kapcsolók adják, például: img2gif -m -d test. Az inkább egyszínű képeknél az -m -d -r 4 vált be nálam. A kis mennyiségű véletlen zaj segít megtörni a sávosodást. Hátránya, hogy a .gif fájl mérete is nő, mivel a véletlen zajt nehéz jól tömöríteni. Szép szürkeárnyalatos képekhez próbálja ki a -f 0 -r kapcsolókat.

paste.exe

A paste eszközzel két nyers formátumú kép illeszthető össze függőlegesen (egymás fölé) vagy vízszintesen (egymás mellé); például két 320x200-as képből 320x400-as vagy 640x200-as kép készíthető. Függőleges illesztéshez a két képnek azonos szélességűnek, vízszinteshez azonos magasságúnak kell lennie.

A paste hívási formája:

```
paste [-h | -v] in_file1.img in_file2.img out_file.img
```

Vízszintes esetben az in_file1.img az in_file2.img-től jobbra jelenik meg. Függőleges esetben az in_file1.img az in_file2.img fölött jelenik meg.

up.exe

Az up a kép méretét függőlegesen, vízszintesen vagy mindkét irányban megduplázza. A méretnövelés pixelduplikálással történik, interpoláció nélkül. Hívási formája:

```
up [-h | -v] in_file.img out_file.img
```

Kapcsoló nélkül az up mindkét irányban megnöveli a kép méretét.

Például ha a test.img egy 320x200-as kép, akkor az

```
up -v test.img test2.img
```

parancs egy 320x400-as felbontású test2.img-et hoz létre.

down.exe

A down az up ellentéte. (Meglepetés!) A kép méretének csökkentésekor a down a megfelelő pixelek átlagolásával állítja elő az új pixelt. Hívási formája:

```
down [-h | -v] infile.img outfile.img
```

Kapcsoló nélkül a down mindkét irányban csökkenti a képet.

Például ha a test egy 320x200-as kép, akkor a

```
down test.img test2.img
```

parancs egy 160x100-as felbontású test2.img-et állít elő.

Fájlformátumok

A fájlformátumokról itt közölt információk azt a célt szolgálják, hogy segítsenek a Vivid formátumaival kompatibilis saját képfeldolgozó segédprogramok írásában. Arra biztatom, hogy kísérletezzen, és ossza meg velünk az érdekes eredményeket.

img – A Vivid kimeneti fájljainak formátuma. A képek pixelenként 24 biten tárolódnak, egyszerű futáshossz-kódolással (run length encoding) a méret csökkentésére. A futáshossz-kódolás úgy működik, hogy az azonos színek ismétlődő sorozatát egy darabszámmal és a csak egyszer kiírt színnel helyettesíti. Az azonos színű „futások” nem nyúlhatnak túl a képsor végén, hogy egyszerűbb legyen soronként dolgozni. A formátum egy 10 bájtos fejlécből és az azt követő képadatokból áll. A fejlécben lévő 16 bites számok a legnagyobb helyiértékű bájttal kezdődnek (big-endian). Ez a formátum kompatibilis az Alias Research által használttal. (Azt hiszem.)

```
<2 bájt>   a kép x mérete
<2 bájt>   a kép y mérete
<2 bájt>   első képsor, általában 0
<2 bájt>   utolsó képsor, általában y méret - 1
<2 bájt>   bitsíkok száma, mindig 24
<képadatok>
```

A képadatok formátuma:

```
<1 bájt>   a következő szín ismétlésszáma
<1 bájt>   kék, 0..255
<1 bájt>   zöld, 0..255
<1 bájt>   vörös, 0..255
```

Ez a kép kitöltéséhez szükséges számú alkalommal ismétlődik. Megjegyzés: a futások nem folytatódnak át egyik képsorról a következőre; ez egyszerűsíti az utófeldolgozást. Felmerült (szia, Ray!), hogy a Vivid egyes élsimítási módjaiban egy plusz képsor kerül a fájlba. Ez idővel talán javítva lesz, de egyelőre az a legjobb, ha mindig a fejléc információit használja, ahelyett hogy a fájl végét keresné.

map – A palettafájlok formátuma. A formátumot a fractinttel való kompatibilitás miatt választottam. A fájl egyszerű ASCII szöveg, 256 sorból áll, minden sor az adott palettabejegyzés vörös, zöld és kék értékét tartalmazza. Az rgb-értékek 0..255 tartományba eső egész számok.

Irodalomjegyzék

Mindazoknak, akiket érdekel a sugárkövetés és működése, nagyon ajánlom az alábbi könyveket. További hivatkozások Glassner könyvének végén, az irodalomjegyzékben találhatók.

- „Computer Graphics, Principles and Practice, 2nd Ed.”, Foley, van Dam, Feiner és Hughes, 1990, ISBN 0-201-12110-7
- „An Introduction to Ray Tracing”, Andrew S. Glassner (szerk.), Academic Press, 1989, ISBN 0-12-286160-4
- „Illumination and Color in Computer Generated Imagery”, Roy Hall, Springer-Verlag, 1989, ISBN 0-387-96774-5

Megjegyzés a fordításhoz: az eredeti dokumentum végén lévő tárgymutató az eredeti oldalszámokra hivatkozik, ezért a fordításból kimaradt. A kódpéldák kulcsszavai (surface, light, studio stb.) a program szintaxisának részei, ezért angolul maradtak; csak a megjegyzések lettek lefordítva.